Ensembl VEP and its reimplementations: a comparison table for Figure 1A

Working document: panel, caption, Discussion fragment, supplementary table, sources and notes

vepyr paper

17 October 2026

How this was made. The cells for fastVEP and vep-rs come from their preprints and from their repositories at the commits listed in Section 4, read on 17 October 2026; field counts are taken from their source code and documentation, not from running the tools. The vepyr cells come from the main text and supplement of this paper. All tables and the panel are generated from one data file (`paper/tool-comparison/build_table.py`); the panel is typeset in Arial to match Figure 1 and exported as PDF, PNG and SVG. Speed is deliberately left out: the published timings use different hardware, VEP versions, VEP modes and annotation scopes, and are not comparable.

Contents

1 Figure 1A: compact comparison **2**

2 Main text: Discussion fragment **3**

3 Supplementary table: full comparison **4**

4 Sources **7**

5 Notes for the authors **8**

1 Figure 1A: compact comparison

→ **MAIN TEXT** Figure 1, first panel (`panel-tools.{pdf,png,svg}`; `panel-tools-nonotes.*` if the footnotes go to the caption). The concordance row is a placeholder until the chromosome 22 comparison with the merged cache has been run for all three reimplementations.

	Ensembl VEP 116.0 `57ea5c5`	fastVEP 0.4.0+6 `5d8004a`	vep-rs 0.4.0 `f642723`	**vepyr** 0.10.0 `3f21520`
Language	Perl	Rust	Rust	Rust, Python
Parallelism	processes	threads	threads	threads
Cache format	Storable, gzip per 1-Mb region	bincode + zstd (transcripts), OSAv2 (annotations)	JSON, gzip optional	Parquet, ZSTD (core and plugins)
Fields filled[a]	80/80	≤37/80	≤71/80 (34/80)[b]	**80/80**
Transcript sets	Ensembl, RefSeq, merged	Ensembl, RefSeq, merged[c]	Ensembl, RefSeq, merged[d]	Ensembl, RefSeq, merged
Declared variant classes	SNV, indel, SV	SNV, indel, SV[e]	SNV, indel, SV[e]	SNV, indel
External annotation sources	yes	yes	yes	yes
Concordance (chr22, merged)	reference	**TODO**	**TODO**	**TODO**
Interfaces	CLI, web, REST	CLI, web, REST	CLI	Python API, SQL, CLI
Evaluation	streaming	streaming	streaming	**lazy, pushdown**
Output	VCF, tab, JSON	VCF, tab, JSON	VCF, tab, JSON, Parquet	Polars DataFrame/ LazyFrame, VCF

[a] Of the 80 `CSQ` fields of VEP 116 `--everything` (Ensembl cache), those the tool can fill, counted from its source code and documentation. [b] 34/80 on the cache converted from VEP, used for all concordance results (no known variants, SIFT or PolyPhen). [c] From Ensembl and RefSeq GFF3 files (merged: both, with `SOURCE`); RefSeq-specific fields are not written. [d] Through a cache converted from the VEP RefSeq or merged cache; `SOURCE` and the RefSeq fields are not written. [e] As declared by the authors; SV annotation not fully validated against VEP.

Caption draft. **(A)** Ensembl VEP and three reimplementations. *Fields filled*: of the 80 `CSQ` fields that VEP 116 writes with `--everything` and the Ensembl cache, those that each tool can fill. *Declared variant classes*: as stated by the authors of each tool. *External annotation sources*: annotation from data outside the core cache (VEP plugins and custom files in VEP, fastSA sources in fastVEP, built-in plugins in vep-rs, plugin manifests in vepyr). *Concordance*: agreement with VEP on chromosome 22 of HG002 with the merged cache, measured with the same inputs for all tools [TODO]. *Evaluation*: streaming tools process the input in fixed batches; in vepyr, annotation is a lazy query whose columns and genomic predicates determine the annotation work. Versions, sources and the full comparison are given in Supplementary Table Sx.

2 Main text: Discussion fragment

→ **MAIN TEXT** Discussion or Conclusion, positioning vepyr against the other reimplementations (about 150 words). Cite the two preprints.

Two Rust reimplementations of VEP have been described recently, fastVEP and vep-rs (Figure 1A). Both replace the VEP command line and report agreement with VEP on consequence terms or on a subset of fields. fastVEP builds transcript models from GFF3 and fills at most 37 of the 80 `--everything` fields; vep-rs reads its own JSON cache, which, when converted from the VEP cache, carries no known variants or protein predictions. vepyr takes a different route. It converts the complete official cache, so that known variants, population frequencies, protein predictions, regulatory features and RefSeq-specific fields are retained; its output is byte-identical to that of VEP for a whole genome, including plugins and transcript selection; and annotation is a lazy DataFrame operation whose cost depends on the query. Its scope is narrower: human GRCh38, small variants and six data-backed plugins. Structural variants, supported by both other tools, are the main missing class; neither tool reports field-level concordance with VEP for them.

3 Supplementary table: full comparison

→ **SUPPLEMENTARY** Supplementary Table Sx: every attribute with the evidence behind the compact cells of Figure 1A. Requested attributes first (parallelism to output formats), then the additional ones.

Attribute	**Ensembl VEP** 116.0 `57ea5c5`	**fastVEP** 0.4.0+6 `5d8004a`	**vep-rs** 0.4.0 `f642723`	**vepyr** 0.10.0 `3f21520`
Parallelism	multi-process (`--fork`)	multi-threaded (rayon, all cores, no thread option); batches of 1,024 records; known-variant lookup and output sequential	multi-threaded (rayon; `--fork` sets threads, default one per CPU up to 32); bounded reader–annotator–writer pipeline	multi-threaded workers over index-defined position windows; output independent of the number of workers
Implementation language	Perl	Rust; web interface in JavaScript	Rust	Rust engine (DataFusion, Arrow); Python API (PyO3)
Cache format and source	official cache: Storable/Sereal `.gz` files per 1-Mb region; optional tabix-indexed known variants	transcripts: own binary cache (bincode + zstd; 388 MB for GRCh38) built from GFF3; annotations (fastSA): OSAv2 (`.osa2`, zstd-compressed chunks; v1 `.osa` still read); VEP cache read only for known variants (`all_vars.gz`)	own JSON cache, one file per chromosome and 1-Mb region (`.json`; `.json.gz` also read), converted from the VEP Storable cache by a Perl script or built from Ensembl GFF3, GTF and FASTA; plugin data in memory-mapped `.vpd` files	Apache Parquet (ZSTD), partitioned by chromosome and entity, with page indexes; converted from the official VEP cache; prebuilt caches on Hugging Face with pinned revisions and manifests
Cache coverage (`CSQ` fields of `--everything`)	reference: 80, 85 and 86 fields (`ensembl`, `refseq` and `merged` caches, release 116)	49 `CSQ` fields, 44 of the 80; at most 37/80 filled (motif fields have no writer, SIFT and PolyPhen always empty); no per-population frequencies, UniProt, `DOMAINS`, `VARIANT_CLASS`	the 80 fields in the header; 9 always empty (motif, `DOMAINS`, `miRNA`, `UNIPARC`, `UNIPROT_ISOFORM`): at most 71/80; converted cache has no known variants or SIFT/PolyPhen: 34/80	all: 80/80, 85/85, 86/86 fields; 42 plugin fields
Functional scope	SNVs, indels, SVs and CNVs; regulatory and motif features; all `--everything` options	SNVs, indels, MNVs, SVs (DEL, DUP, INV, CNV, BND, INS, STR); mitochondria; ACMG classification; `--everything` accepted without effect; regulatory annotation not connected	SNVs, indels, MNVs, SVs (DEL, INS, DUP, INV, CNV, BND, mobile elements); mitochondria; no regulatory annotation; `--custom`, `--mirna`, `--hgvsg` accepted without effect	SNVs and short indels; regulatory and motif features; `--everything` always on; no SVs
Concordance with VEP: published evidence and metric	reference implementation	exact match of 22 of 24 fields on 2,920 transcript–allele pairs of the 173-variant VEP example (VEP 115.1, `--gff` mode); consequences and HGVS 99.49–100% on ClinVar (6,600 variants) and HG002 (1 in 200) samples; pairs present in one tool only not scored; 9 deliberate divergences; SVs not compared	F1 over (location, allele, feature, consequence set) tuples, default options: $\geq$0.999974 on 2.6×10^8 tuples (VEP 115.2, preprint), 0.9999984 on 1.2×10^{10} tuples (VEP 116.2, repository); all formats field for field on test corpora; SVs: F1 0.90–0.98 on chromosome 21 sets, no field comparison; not byte-identical	strict md5 of all record lines identical to VEP 116.0 on HG002 chr1–22 (4,096,123 records): 3 cache types, 5 plugins + ClinVar, 7 transcript-selection profiles (chr1, chr22); 205 tests ported from VEP
Concordance on chr22, merged cache	reference	**TODO**	**TODO**	**TODO**

Attribute	**Ensembl VEP** 116.0 `57ea5c5`	**fastVEP** 0.4.0+6 `5d8004a`	**vep-rs** 0.4.0 `f642723`	**vepyr** 0.10.0 `3f21520`
User interfaces	CLI; web interface; REST API	CLI; web interface and REST API (`fastvep-web`); release binaries (Linux x86-64, macOS arm64)	CLI with VEP option names (`--json_cache` required); Docker image (amd64)	Python API (Polars) and SQL; CLI with VEP option names; nf-core module and subworkflow
Lazy evaluation and streaming	streaming in buffers (`--buffer_size`, default 5,000); no lazy evaluation or pushdown	streaming in batches of 1,024 records; whole transcript set loaded (subset if the GFF3 is indexed); no region selection or pushdown	streaming in buffers through a bounded pipeline; transcripts loaded per chromosome, known variants loaded in full; no region selection or pushdown	lazy Polars source: projection pushdown (optional annotation work), region pushdown through the VCF index, row limit; streaming sinks
Output formats	VEP default (tab-delimited), VCF (`CSQ`), tab, JSON; HTML statistics	VCF (`CSQ` and `FV_*` fields), tab (17 columns), JSON (VEP and Nirvana style)	VEP default, tab, VCF, JSON; Parquet through an external DuckDB process	Polars `LazyFrame`/`DataFrame` with typed columns; VCF (`CSQ`; plain, gzip, BGZF); Parquet and other formats through Polars sinks
Transcript sets	Ensembl, RefSeq, merged; GFF/GTF	Ensembl, RefSeq or merged from GFF3 files (`--gff3` repeated, per-transcript `SOURCE`); RefSeq-specific fields (`REFSEQ_MATCH`, `BAM_EDIT`, ...) not written	Ensembl; RefSeq and merged through a cache converted from the corresponding VEP cache (`--refseq` and `--merged` have no effect), but `SOURCE` and the RefSeq fields (`REFSEQ_MATCH`, `BAM_EDIT`, ...) are not written and the RefSeq BAM-edit correction is not implemented (open items of the project's parity matrix); no RefSeq concordance reported	Ensembl, RefSeq, merged
Transcript selection (`--pick` etc.)	`--pick`, `--per_gene`, `--flag_pick`, `--most_severe` and others	`--pick` family (since 0.4.0)	`--pick` family; `--pick_order` accepted without effect	`pick`, `pick_allele`, `per_gene`, `pick_allele_gene`, 3 `flag_pick` variants
External annotation sources	83 Perl plugin modules (`VEP_plugins`, release 116); `--custom` (VCF, BED, GFF/GTF, bigWig)	fastSA: 18 built-in sources (ClinVar, gnomAD, dbNSFP, SpliceAI, AlphaMissense, REVEL, ...) and custom VCF/BED; no VEP plugins	11 built-in plugins (CADD, REVEL, AlphaMissense, SpliceAI, LoFTEE, ...); C-ABI shared-library plugins; no `--custom`	declarative TOML manifests (TSV, CSV, Parquet, VCF, BED; SQL ingestion): CADD, SpliceAI, AlphaMissense, dbNSFP, PhenotypeOrthologous, ClinVar; no executable plugins
Filtering of results (`filter_vep`)	`filter_vep` (own filter language)	`fastvep filter`	none	Polars expressions or SQL; 18 `filter_vep` queries reproduced exactly
Distribution	GitHub (`INSTALL.pl`), Docker, Singularity, Bioconda; nf-core module	GitHub releases; conda recipe not on Bioconda; no container image	GitHub releases; Docker (ghcr.io)	PyPI wheels (Linux, macOS, Windows), Bioconda, nf-core
Species, assemblies, VEP releases	all Ensembl species; GRCh37 and GRCh38	any organism with GFF3 and FASTA (5 benchmarked); GRCh37, GRCh38	*Homo sapiens* GRCh37 and GRCh38; VEP 115.2 (preprint), 116.2 (repository)	*Homo sapiens* GRCh38; VEP 115.2 and 116.0

Licence	Apache-2.0	Apache-2.0	Apache-2.0	Apache-2.0

Versions and git commits (also in the column headers): Ensembl VEP 116.0 (`ensembl-vep` tag `release/116.0`, `57ea5c5`; `VEP_plugins` branch `release/116`, `59b34e5`); fastVEP default branch at `5d8004a` (11 October 2026, six commits after tag `v0.4.0`, `d728c46`) and preprint version 2; vep-rs tag `v0.4.0` (`f642723`, 16 October 2026) and preprint version 1; vepyr tag `0.10.0` (`3f21520`), validated with `0.9.2` (`844464c`). Field counts refer to the 80 `CSQ` fields of VEP 116 `--everything` with the Ensembl cache (85 with RefSeq, 86 with the merged cache) and were obtained from the source code and documentation of each tool.

4 Sources

→ **SUPPLEMENTARY** References for the table and the Discussion; add to `references.bib` after checking the final versions.

- **Ensembl VEP.** McLaren W, et al. The Ensembl Variant Effect Predictor. *Genome Biol* 17:122 (2016). `https://github.com/Ensembl/ensembl-vep` (tag `release/116.0`, commit `57ea5c52340a`); `https://github.com/Ensembl/VEP_plugins` (branch `release/116`, commit `59b34e58b41c`, 83 plugin modules).
- **fastVEP.** Huang K. fastVEP: a high-performance variant effect predictor implemented in Rust. *bioRxiv* doi:10.64898/2026.04.14.718452, version 2 (9 August 2026; version 1, 16 April 2026, was titled "fastVEP: A Fast, Comprehensive Variant Effect Predictor Written in Rust"). `https://github.com/Huang-lab/fastVEP`, commit `5d8004a72fb7` (11 October 2026; tag `v0.4.0` is `d728c46fbd67`). Licence Apache-2.0.
- **vep-rs.** Porter M, Borkowski R. vep-rs: high-throughput Rust variant annotation with population-scale concordance to Ensembl VEP. *bioRxiv* doi:10.64898/2026.09.22.753614, version 1 (28 September 2026); formatted for *Bioinformatics*. `https://github.com/natera-open-source/vep-rs`, tag `v0.4.0`, commit `f64272326dee` (16 October 2026). Licence Apache-2.0.
- **vepyr.** This paper: Methods; Supplementary sections Scope, Plugins, Pushdowns and Validation. `https://github.com/biodatageeks/vepyr`, tag `0.10.0` (commit `3f2152004c86`); validation with tag `0.9.2` (commit `844464c6b580`).

Evidence for the compact cells

fastVEP, fields. `DEFAULT_CSQ_FIELDS` has 49 names (`crates/fastvep-io/src/output.rs:472--522`), 44 of them among the 80; the five motif fields have no writer (`output.rs:41--46`) and `sift`/`polyphen` are always `None` (`crates/fastvep-annotate/src/lib.rs:428--429`): at most 37/80. Missing: `VARIANT_CLASS`, UniProt fields, `GENE_PHENO`, `DOMAINS`, `miRNA`, all 28 population frequency fields and `MAX_AF(_POPS)`. `--everything`, `--symbol`, `--canonical` and `--buffer-size` are parsed and discarded (`crates/fastvep-cli/src/main.rs:343--353`).

fastVEP, concordance. README ("22 of the 24 compared CSQ fields agree on 100% of 2,920 shared (allele, transcript) pairs from the 173-variant VEP example set"; VEP v115.1 "`--gff` mode"); ClinVar sample of 6,600 variants and 1-in-200 HG002 sample: consequence set 99.96%/100%, HGVSc 99.97%/99.98%, HGVSp 99.49%/99.99%; rows present in one tool only are not scored. No SV is compared with VEP: the validation datasets exclude SVs, which are covered by unit tests only.

vep-rs, fields. The `--everything` header lists the 80 fields; regulatory and motif annotation is not implemented, `DOMAINS` is never populated, `miRNA`, `UNIPARC`, `UNIPROT_ISOFORM` and `HGVSg` are always empty (`docs/cli-reference.md`, "Compatibility with Perl VEP"): at most 71/80. "A converted cache carries no variation data and no SIFT or PolyPhen matrices" (`docs/cache-setup.md`), which empties a further 37 fields: 34/80.

vep-rs, transcript sets. `--refseq` and `--merged` "selects no transcript set (the transcript set is the `--json_cache` contents)" (`docs/cli-reference.md`); README: "accepted but not implemented". The Perl converter accepts any Storable cache directory, so RefSeq or merged transcripts can be loaded, but the field table has no `REFSEQ_MATCH`, `REFSEQ_OFFSET`, `GIVEN_REF`, `USED_REF` or `BAM_EDIT`, and `SOURCE` has no writer (it falls through to the plugin lookup and is empty; `crates/vep-io/src/output/fields.rs`). In the project's parity matrix (`tests/parity/matrix.tsv`) the rows `refseq`, `merged`, `refseq-merged-cache-selection`, `refseq-match-bam-edit-source-columns` and `bam-refseq-model-correction` are all `open`. All published concordance uses the converted Ensembl cache.

vep-rs, concordance. Preprint: F1 over (location, allele, feature, consequence set) tuples, $\geq$0.999974 on 2.6×10^8 tuples from six datasets, VEP 115.2 with default options; "It does not certify VEP's other output fields, and no VEP plugin enters any measurement". Repository: F1 0.9999984 on 1.2×10^{10} tuples (VEP 116.2). Outputs compared field for field on golden corpora with default flags; JSON "not byte-identical". SVs: raw F1 0.975 (GRCh37) and 0.910 (GRCh38) on 16 files per assembly (chromosome 21 and synthetic), "no per-column comparator".

5 Notes for the authors

NOT FOR THE PAPER Observations from reading the two preprints and repositories.

1. **Make the field counts empirical.** The counts (≤37/80, ≤71/80, 34/80) are read from source code and documentation. Before submission, run both tools on HG002 chromosome 22 with their closest equivalent of `--everything` and count the non-empty values per `CSQ` field; the table then rests on measured output, which is harder to dispute.

2. **Speed claims of the others are moving targets.** fastVEP version 2 withdrew the "up to 130×" and "∼191×" figures of version 1; it now reports 23.4× single-threaded on HG002 against VEP 115.1 run in `--gff` mode (not with the cache), with the VEP genome time summed over per-chromosome runs. vep-rs reports 78–284× against VEP 115.2 with `--fork 16` on AWS instances, and 5–14× against fastVEP. This supports leaving speed out of the comparison table and benchmarking on matched scope instead.

3. **vep-rs measured fastVEP.** On the same inputs, the vep-rs preprint reports fastVEP F1 of 0.762–0.970 on the six SNV/indel datasets and 0.13–0.15 on SVs, largely because its loader drops transcripts that the VEP cache holds. This is a third-party measurement; cite it only with attribution, if at all.

4. **Reproducing VEP's defects.** vep-rs documents behaviour of VEP 115 that it calls defects and does not reproduce (co-emission of `start_lost` and `start_retained_variant`, batch-dependent transcript selection for very large SVs, SV annotation against transcripts on another chromosome). Byte identity means vepyr reproduces whatever VEP writes for small variants, including the first of these. A reviewer may raise it; the position to state is that the goal is interchangeability with existing VEP outputs, filters and validated pipelines, and that any correction would have to be an explicit option.

5. **Scale and diversity of the concordance test.** vep-rs compares 1.2×10^{10} tuples across ClinVar, gnomAD and 1000 Genomes, but on consequence terms only; vepyr compares all fields byte for byte, but on one genome (4.1 million records, three caches, plugins, transcript selection). A reviewer may ask for more variant diversity. A genome-wide ClinVar GRCh38 run (rich in indels and pathogenic coding variants) with the same strict md5 comparison would close that gap, before submission or in revision.

6. **Structural variants.** fastVEP supports SVs without any comparison with VEP; vep-rs compares SV consequence terms (not fields) on chromosome 21 and synthetic files. The table states vepyr's lack of SV support explicitly; if SV support is added in revision, it should be validated by the same standard before it enters the table.

7. **Moving repositories.** Both repositories changed substantially after their preprints (fastVEP: `--pick`, field counts; vep-rs: VEP 116.2, whole-genome concordance). The table describes the repository state on 17 October 2026; update the cells and versions before submission (`build_table.py`).

8. **vep-rs and VEP releases.** vep-rs 0.4.0 targets VEP 116.2; vepyr is validated against 116.0 (whole genome) and 116.2 (ported tests). The release-116 caches are the same, but the version difference should be stated if the two are compared directly.